

Graphs

Tarjan Strongly Connected Components

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
int disc[MAXN], low[MAXN], timer;
bool inStack[MAXN];
stack<int> st;
vector<vector<int>> sccs;

void tarjan(int u) {
    disc[u] = low[u] = ++timer;
    st.push(u);
    inStack[u] = true;

    for (int v : adj[u]) {
        if (!disc[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStack[v]) {
            low[u] = min(low[u], disc[v]);
        }
    }

    if (disc[u] == low[u]) {
        vector<int> scc;
        int v;
        do {
            v = st.top();
            st.pop();
            inStack[v] = false;
            scc.push_back(v);
        } while (v != u);
        sccs.push_back(scc);
    }
}

void findSCCs(int n) {
    memset(disc, 0, sizeof disc);
    memset(low, 0, sizeof low);
    memset(inStack, false, sizeof inStack);
    while (!st.empty())
        st.pop();
    sccs.clear();
    timer = 0;
}
```

```

    for (int i = 0; i < n; ++i)
        if (!disc[i])
            tarjan(i);
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    // Example usage
    int n = 5;
    // Add edges manually or via input
    adj[1].push_back(0);
    adj[0].push_back(2);
    adj[2].push_back(1);
    adj[0].push_back(3);
    adj[3].push_back(4);

    findSCCs(n);

    // Output SCCs
    for (auto &scc : sccs) {
        for (int x : scc)
            cout << x << " ";
        cout << "\n";
    }

    return 0;
}

```

Bipartite Graph Maximum Matching

```

// Note: If the matching found by the Hopcroft-Karp algorithm is not a perfect
// matching, it is guaranteed that the graph does not have one.

#define PROBLEM "https://judge.yosupo.jp/problem/bipartitematching"

#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("avx,avx2,sse,sse2,sse3,sse4,popcnt")

#include <bits/stdc++.h>

namespace IO {
constexpr bool UNSAFE = false;
constexpr int GLOB_BUF_SIZE = 1 << 15;

```

```

#ifndef DEBUG
#define CHANGE_DEFAULT_STREAMS
static struct FastInput {
    FastInput() {
        if constexpr (UNSAFE) {
            chars_read = fread(buf, 1, BUF_SIZE, in);
            buf_pos = 0;
            buf[0] = (chars_read == 0 ? -1 : buf[0]);
        }
    }
    static constexpr int BUF_SIZE = GLOB_BUF_SIZE;
    char buf[BUF_SIZE];
    size_t chars_read = 0;
    size_t buf_pos = 0;
    FILE *in = stdin;
    char cur = 0;
    inline char get_char() {
        if constexpr (!UNSAFE) {
            if (buf_pos >= chars_read) {
                chars_read = fread(buf, 1, BUF_SIZE, in);
                buf_pos = 0;
                buf[0] = (chars_read == 0 ? -1 : buf[0]);
            }
        }
        return cur = buf[buf_pos++];
    }
    template <typename T> inline FastInput *tie(T) { return this; }
    inline void sync_with_stdio(bool) {}
    inline explicit operator bool() { return cur != -1; }
    inline static bool is_blank(char c) { return c <= ' '; }
    inline bool skip_blanks() {
        while (is_blank(cur) && cur != -1)
            get_char();
        return cur != -1;
    }
    inline FastInput &operator>>(char &c) {
        skip_blanks();
        c = cur;
        return *this;
    }
    inline FastInput &operator>>(std::string &s) {
        if (skip_blanks()) {
            s.clear();
            do {
                s += cur;
            } while (!is_blank(get_char()));
        }
        return *this;
    }
    template <typename T> inline FastInput &read_integer(T &n) {
        // unsafe, doesn't check that characters are actually digits
        n = 0;
        if (skip_blanks()) {
            int sign = +1;

```

```

    if (cur == '-') {
        sign = -1;
        get_char();
    }
    do {
        n += n + (n << 3) + cur - '0';
    } while (!is_blank(get_char()));
    n *= sign;
}
return *this;
}
template <typename T>
inline typename std::enable_if<std::is_integral<T>::value, FastInput &>::type
operator>>(T &n) {
    return read_integer(n);
}
#ifdef _WIN32 || defined(_WIN64)
inline FastInput &operator>>(__int128 &n) { return read_integer(n); }
#endif
template <typename T>
inline typename std::enable_if<std::is_floating_point<T>::value,
                               FastInput &>::type
operator>>(T &n) {
    // not sure if really fast, for compatibility only
    n = 0;
    if (skip_blanks()) {
        std::string s;
        (*this) >> s;
        sscanf(s.c_str(), "%lf", &n);
    }
    return *this;
}
} fast_input;
#define cin IO::fast_input
static struct FastOutput {
    static constexpr int BUF_SIZE = GLOB_BUF_SIZE;
    char buf[BUF_SIZE];
    size_t buf_pos = 0;
    static constexpr int TMP_SIZE = GLOB_BUF_SIZE;
    char tmp[TMP_SIZE];
    FILE *out = stdout;
    inline void put_char(char c) {
        buf[buf_pos++] = c;
        if (buf_pos == BUF_SIZE) {
            fwrite(buf, 1, buf_pos, out);
            buf_pos = 0;
        }
    }
} ~FastOutput() { fwrite(buf, 1, buf_pos, out); }
inline FastOutput &operator<<(char c) {
    put_char(c);
    return *this;
}
inline FastOutput &operator<<(const char *s) {

```

```

    while (*s)
        put_char(*s++);
    return *this;
}

inline FastOutput &operator<<(const std::string &s) {
    for (auto x : s)
        put_char(x);
    return *this;
}

template <typename T> inline char *integer_to_string(T n) {
    // beware of TMP_SIZE
    char *p = tmp + TMP_SIZE - 1;
    if (n == 0)
        *--p = '0';
    else {
        bool is_negative = false;
        if (n < 0) {
            is_negative = true;
            n = -n;
        }
        while (n > 0) {
            *--p = (char)('0' + n % 10);
            n /= 10;
        }
        if (is_negative)
            *--p = '-';
    }
    return p;
}

template <typename T>
inline typename std::enable_if<std::is_integral<T>::value, char *>::type
stringify(T n) {
    return integer_to_string(n);
}

#ifdef _WIN32 || defined(_WIN64)
    inline char *stringify(__int128 n) { return integer_to_string(n); }
#endif

template <typename T>
inline typename std::enable_if<std::is_floating_point<T>::value, char *>::type
stringify(T n) {
    sprintf(tmp, "%.17f", n);
    return tmp;
}

template <typename T> inline FastOutput &operator<<(const T &n) {
    auto p = stringify(n);
    for (; *p != 0; p++)
        put_char(*p);
    return *this;
}

} fast_output;
#define cout IO::fast_output
#endif
} // namespace IO

```

```

struct Random : std::mt19937 {
    using std::mt19937::mt19937;
    using std::mt19937::operator();
    static int64_t gen_seed() {
        return std::chrono::steady_clock::now().time_since_epoch().count();
    }
    Random() : std::mt19937(gen_seed()) {}
    template <class Int>
    auto operator()(Int a, Int b)
        -> std::enable_if_t<std::is_integral_v<Int>, Int> {
        return std::uniform_int_distribution<Int>(a, b)(*this);
    }
    template <class Int>
    auto operator()(Int a) -> std::enable_if_t<std::is_integral_v<Int>, Int> {
        return std::uniform_int_distribution<Int>(0, a - 1)(*this);
    }
    template <class Real>
    auto operator()(Real a, Real b)
        -> std::enable_if_t<std::is_floating_point_v<Real>, Real> {
        return std::uniform_real_distribution<Real>(a, b)(*this);
    }
};

template <bool ToShuffle = false> struct bipartite_matching {
    int n_left, n_right, flow = 0;
    std::vector<std::vector<int>> g;
    std::vector<int> match_from_left, match_from_right;

    bipartite_matching(int _n_left, int _n_right)
        : n_left(_n_left), n_right(_n_right), g(_n_left),
          match_from_left(_n_left, -1), match_from_right(_n_right, -1),
          dist(_n_left) {}

    void add(int u, int v) { g[u].push_back(v); }

    std::vector<int> dist;

    void bfs() {
        std::queue<int> q;
        for (int u = 0; u < n_left; ++u) {
            if (!match_from_left[u])
                q.push(u), dist[u] = 0;
            else
                dist[u] = -1;
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto v : g[u])
                if (~match_from_right[v] && !~dist[match_from_right[v]]) {
                    dist[match_from_right[v]] = dist[u] + 1;
                    q.push(match_from_right[v]);
                }
        }
    }
};

```

```

}

bool dfs(int u) {
    for (auto v : g[u])
        if (!~match_from_right[v]) {
            match_from_left[u] = v, match_from_right[v] = u;
            return true;
        }
    for (auto v : g[u])
        if (dist[match_from_right[v]] == dist[u] + 1 &&
            dfs(match_from_right[v])) {
            match_from_left[u] = v, match_from_right[v] = u;
            return true;
        }
    return false;
}

int get_max_matching() {
    if constexpr (ToShuffle) {
        Random rng;
        for (int i = 0; i < n_left; ++i)
            std::random_shuffle(std::begin(g[i]), std::end(g[i]), rng);
    }
    while (true) {
        bfs();
        int augment = 0;
        for (int u = 0; u < n_left; ++u)
            if (!~match_from_left[u])
                augment += dfs(u);
        if (!augment)
            break;
        flow += augment;
    }
    return flow;
}

std::pair<std::vector<int>, std::vector<int>> minimum_vertex_cover() {
    std::vector<int> L, R;
    for (int u = 0; u < n_left; ++u) {
        if (!~dist[u])
            L.push_back(u);
        else if (~match_from_left[u])
            R.push_back(match_from_left[u]);
    }
    return {L, R};
}

std::vector<std::pair<int, int>> get_edges() {
    std::vector<std::pair<int, int>> ans;
    for (int u = 0; u < n_left; ++u)
        if (match_from_left[u] != -1)
            ans.emplace_back(u, match_from_left[u]);
    return ans;
}

```

```
};

int main() {
    int n_left, n_right, m;
    cin >> n_left >> n_right >> m;
    bipartite_matching<false> matching(n_left, n_right);
    while (m--) {
        int u, v;
        cin >> u >> v;
        matching.add(u, v);
    }
    cout << matching.get_max_matching() << '\n';
    for (auto [u, v] : matching.get_edges())
        cout << u << ' ' << v << '\n';
}
```

Eulerian Cycle Or Path

```
// C++ program to check whether a graph is
// Eulerian Path, Eulerian Circuit, or neither
#include <bits/stdc++.h>
using namespace std;

// DFS to check connectivity, excluding zero-degree vertices
void dfs(int node, vector<int> adj[], vector<bool> &visited) {
    visited[node] = true;

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}

// Function to check Eulerian Path or Circuit
// 2 if Cycle, 1 if Path, 0 if neither
int isEulerCircuit(int v, vector<int> adj[]) {
    vector<bool> visited(v, false);

    // Find first vertex with non-zero degree
    int start = -1;
    for (int i = 0; i < v; i++) {
        if (adj[i].size() > 0) {
            start = i;
            break;
        }
    }
}
```

```
// No edges: graph is trivially Eulerian
if (start == -1) {
    return 2;
}

// DFS from the first non-zero degree vertex
dfs(start, adj, visited);

// Check if all non-zero degree vertices are connected
for (int i = 0; i < v; i++) {
    if (adj[i].size() > 0 && !visited[i]) {
        return 0; // Not connected
    }
}

// Count vertices with odd degree
int odd = 0;
for (int i = 0; i < v; i++) {
    if (adj[i].size() % 2 != 0) {
        odd++;
    }
}

// Apply Eulerian rules
if (odd == 0) {
    return 2;
} else if (odd == 2) {
    return 1;
} else {
    return 0;
}
}

// Driver code
int main() {

    int v = 5;
    vector<int> adj[5] = {{1, 2, 3}, {0, 2}, {1, 0}, {0, 4}, {3}};

    cout << isEulerCircuit(v, adj);

    return 0;
}
```

Hamiltonian Path

```
// C++ program for the above approach

#include <bits/stdc++.h>
using namespace std;
const int N = 5;

// Function to check whether there
// exists a Hamiltonian Path or not
bool Hamiltonian_path(vector<vector<int>> &adj, int N) {
    int dp[N][(1 << N)];

    // Initialize the table
    memset(dp, 0, sizeof dp);

    // Set all dp[i][(1 << i)] to
    // true
    for (int i = 0; i < N; i++)
        dp[i][(1 << i)] = true;

    // Iterate over each subset
    // of nodes
    for (int i = 0; i < (1 << N); i++) {

        for (int j = 0; j < N; j++) {

            // If the jth nodes is included
            // in the current subset
            if (i & (1 << j)) {

                // Find K, neighbour of j
                // also present in the
                // current subset
                for (int k = 0; k < N; k++) {

                    if (i & (1 << k) && adj[k][j] && j != k && dp[k][i ^ (1 << j)]) {

                        // Update dp[j][i]
                        // to true
                        dp[j][i] = true;
                        break;
                    }
                }
            }
        }
    }

    // Traverse the vertices
    for (int i = 0; i < N; i++) {

        // Hamiltonian Path exists
        if (dp[i][(1 << N) - 1])
            return true;
    }
}
```

```

    // Otherwise, return false
    return false;
}

// Driver Code
int main() {

    // Input
    vector<vector<int>> adj = {
        {0, 1, 1, 1, 0}, {1, 0, 1, 0, 1}, {1, 1, 0, 1, 1}, {1, 0, 1, 0, 0}};
    int N = adj.size();

    // Function Call
    if (Hamiltonian_path(adj, N))
        cout << "YES";
    else
        cout << "NO";

    return 0;
}

```

Hungarian

```

#include <cassert>
#include <iostream>
#include <limits>
#include <vector>

using namespace std;

template <typename T, typename U> using Pair = std::pair<T, U>;
template <typename T> using Vector = std::vector<T>;

template <typename T> using NumericLimits = std::numeric_limits<T>;

/**
 * @brief Checks if b < a
 *
 * Sets a = min(a, b)
 * @param a The first parameter to check
 * @param b The second parameter to check
 * @tparam The type to perform the check on
 * @return true if b < a
 */
template <typename T> constexpr bool ckmin(T &a, const T &b) {
    return b < a ? a = b, true : false;
}

```

```

}

/**
 * @brief Performs the Hungarian algorithm.
 *
 * Given J jobs and W workers (J <= W), computes the minimum cost to assign each
 * prefix of jobs to distinct workers.
 *
 * @tparam T a type large enough to represent integers on the order of J *
 * max(|C|)
 * @param C a matrix of dimensions JxW such that C[j][w] = cost to assign j-th
 * job to w-th worker (possibly negative)
 *
 * @return a vector of length J, with the j-th entry equaling the minimum cost
 * to assign the first (j+1) jobs to distinct workers
 */
template <typename T> Vector<T> hungarian(const Vector<Vector<T>> &C) {
    const int J = static_cast<int>(C.size());
    const int W = static_cast<int>(C[0].size());
    assert(J <= W);
    // job[w] = job assigned to w-th worker, or -1 if no job assigned
    // note: a W-th worker was added for convenience
    Vector<int> job(W + 1, -1);
    Vector<T> ys(J);
    Vector<T> yt(W + 1); // potentials
    // -yt[W] will equal the sum of all deltas
    Vector<T> answers;
    const T inf = NumericLimits<T>::max();
    for (int jCur = 0; jCur < J; ++jCur) { // assign jCur-th job
        int wCur = W;
        job[wCur] = jCur;
        // min reduced cost over edges from Z to worker w
        Vector<T> minTo(W + 1, inf);
        Vector<int> prev(W + 1, -1); // previous worker on alternating path
        Vector<bool> inZ(W + 1);    // whether worker is in Z
        while (job[wCur] != -1) {    // runs at most jCur + 1 times
            inZ[wCur] = true;
            const int j = job[wCur];
            T delta = inf;
            int wNext;
            for (int w = 0; w < W; ++w) {
                if (!inZ[w]) {
                    if (ckmin(minTo[w], C[j][w] - ys[j] - yt[w]))
                        prev[w] = wCur;
                    if (ckmin(delta, minTo[w]))
                        wNext = w;
                }
            }
        }
        // delta will always be nonnegative,
        // except possibly during the first time this loop runs
        // if any entries of C[jCur] are negative
        for (int w = 0; w <= W; ++w) {
            if (inZ[w]) {
                ys[job[w]] += delta;
            }
        }
    }
}

```

```

        yt[w] -= delta;
    } else {
        minTo[w] -= delta;
    }
}
wCur = wNext;
}
// update assignments along alternating path
for (int w; wCur != W; wCur = w)
    job[wCur] = job[w = prev[wCur]];
answers.push_back(-yt[W]);
}
return answers;
}

/**
 * @brief Performs a sanity check for the Hungarian algorithm.
 *
 * Sanity check: https://en.wikipedia.org/wiki/Hungarian\_algorithm#Example
 * First job (5):
 *   clean bathroom: Bob -> 5
 * First + second jobs (9):
 *   clean bathroom: Bob -> 5
 *   sweep floors: Alice -> 4
 * First + second + third jobs (15):
 *   clean bathroom: Alice -> 8
 *   sweep floors: Carol -> 4
 *   wash windows: Bob -> 3
 */
void sanityCheckHungarian() {
    Vector<Vector<int>> costs{{8, 5, 9}, {4, 2, 4}, {7, 3, 8}};
    assert((hungarian(costs) == Vector<int>{5, 9, 15}));
    std::println(stderr, "Sanity check passed.");
}

/**
 * @brief solves https://open.kattis.com/problems/cordonbleu
 */
void cordonBleu() {
    int N;
    int M;
    std::cin >> N >> M;
    Vector<Pair<int, int>> B(N);
    Vector<Pair<int, int>> C(M);
    Vector<Pair<int, int>> bottles(N);
    Vector<Pair<int, int>> couriers(M);
    for (const Pair<int, int> &b : bottles)
        std::cin >> b.first >> b.second;
    for (const Pair<int, int> &c : couriers)
        std::cin >> c.first >> c.second;
    Pair<int, int> rest;
    std::cin >> rest.first >> rest.second;
    Vector<Vector<int>> costs(N, Vector<int>(N + M - 1));
    auto dist = [&](const Pair<int, int> &x, const Pair<int, int> &y) -> int {

```

```

    return std::abs(x.first - y.first) + std::abs(x.second - y.second);
};
for (int b = 0; b < N; ++b) {
    for (int c = 0; c < M; ++c) // courier -> bottle -> restaurant
        costs[b][c] = dist(couriers[c], bottles[b]) + dist(bottles[b], rest);
    for (int _ = 0; _ < N - 1; ++_) // restaurant -> bottle -> restaurant
        costs[b][_ + M] = 2 * dist(bottles[b], rest);
}
std::println(hungarian(costs).back());
}

/**
 * @brief Entry point into the program.
 *
 * @return The return code of the program.
 */
int main() {
    sanityCheckHungarian();
    cordonBleu();
}

```

Hungarian-Shortest-Paths

```

#include <cassert>
#include <limits>
#include <vector>

using namespace std;

// The implementation of the Hungarian algorithm can be refactored to highlight
// this connection. The worker potentials, represented by h in this new
// approach, are equivalent to the potentials y from the previous solution, with
// only a constant offset. For cases where the graph is sparse (meaning there
// are only M allowed job-worker pairs), you can optimize the algorithm to run
// in  $O(J \cdot M + J^2 \cdot \log(W))$  time. This is achieved by using a Fibonacci heap to find
// the next worker to process, rather than iterating through all W workers. This
// modification significantly improves performance by efficiently identifying
// the worker with the minimum distance.

template <typename T, typename U> using Pair = std::pair<T, U>;
template <typename T> using Vector = std::vector<T>;

template <typename T> using NumericLimits = std::numeric_limits<T>;

/**
 * @brief Checks if b < a
 *

```

```

* Sets a = min(a, b)
* @param a The first parameter to check
* @param b The second parameter to check
* @tparam The type to perform the check on
* @return true if b < a
*/
template <typename T> constexpr bool ckmin(T &a, const T &b) {
    return b < a ? a = b, true : false;
}

template <typename T> Vector<T> hungarian(const Vector<Vector<T>> &C) {
    const int J = static_cast<int>(C.size());
    const int W = static_cast<int>(C[0].size());
    assert(J <= W);
    // job[w] = job assigned to w-th worker, or -1 if no job assigned
    // note: a W-th worker was added for convenience
    Vector<int> job(W + 1, -1);
    Vector<T> h(W); // Johnson potentials
    Vector<T> answers;
    T ansCur = 0;
    const T inf = std::numeric_limits<T>::max();
    // assign jCur-th job using Dijkstra with potentials
    for (int jCur = 0; jCur < J; ++jCur) {
        int wCur = W; // unvisited worker with minimum distance
        job[wCur] = jCur;
        Vector<T> dist(W + 1, inf); // Johnson-reduced distances
        dist[W] = 0;
        Vector<bool> vis(W + 1); // whether visited yet
        Vector<int> prev(W + 1, -1); // previous worker on shortest path
        while (job[wCur] != -1) { // Dijkstra step: pop min worker from heap
            T minDist = inf;
            vis[wCur] = true;
            int wNext = -1; // next unvisited worker with minimum distance
            // consider extending shortest path by wCur -> job[wCur] -> w
            for (int w = 0; w < W; ++w) {
                if (!vis[w]) {
                    // sum of reduced edge weights wCur -> job[wCur] -> w
                    T edge = C[job[wCur]][w] - h[w];
                    if (wCur != W) {
                        edge -= C[job[wCur]][wCur] - h[wCur];
                        assert(edge >= 0); // consequence of Johnson potentials
                    }
                    if (ckmin(dist[w], dist[wCur] + edge))
                        prev[w] = wCur;
                    if (ckmin(minDist, dist[w]))
                        wNext = w;
                }
            }
            wCur = wNext;
        }
        for (int w = 0; w < W; ++w) { // update potentials
            ckmin(dist[w], dist[wCur]);
            h[w] += dist[w];
        }
    }
}

```

```

    ansCur += h[wCur];
    for (int w; wCur != W; wCur = w)
        job[wCur] = job[w = prev[wCur]];
    answers.push_back(ansCur);
}
return answers;
}

```

Johnsons ALogrithm

```

#include <iostream>
#include <limits>
#include <vector>

#define INF std::numeric_limits<int>::max()

using namespace std;

// Function to find the vertex with the minimum distance
// that has not yet been included in the shortest path tree
int Min_Distance(const vector<int> &dist, const vector<bool> &visited) {
    int min = INF, min_index;
    for (int v = 0; v < dist.size(); ++v) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to perform Dijkstra's algorithm on the modified graph
void Dijkstra_Algorithm(const vector<vector<int>> &graph,
                        const vector<vector<int>> &altered_graph, int source) {
    int V = graph.size(); // Number of vertices
    vector<int> dist(V, INF); // Distance from source to each vertex
    vector<bool> visited(V, false); // Track visited vertices

    dist[source] = 0; // Distance to source itself is 0

    // Compute shortest path for all vertices
    for (int count = 0; count < V - 1; ++count) {
        // Select the vertex with the minimum distance that hasn't been visited
        int u = Min_Distance(dist, visited);
        visited[u] = true; // Mark this vertex as visited

        // Update the distance value of the adjacent vertices of the selected vertex
    }
}

```

```

    for (int v = 0; v < V; ++v) {
        if (!visited[v] && graph[u][v] != 0 && dist[u] != INF &&
            dist[u] + altered_graph[u][v] < dist[v]) {
            dist[v] = dist[u] + altered_graph[u][v];
        }
    }
}

// Print the shortest distances from the source
cout << "Shortest Distance from vertex " << source << ":\n";
for (int i = 0; i < V; ++i) {
    cout << "Vertex " << i << ": "
        << (dist[i] == INF ? "INF" : to_string(dist[i])) << endl;
}
}

// Function to perform Bellman-Ford algorithm to find shortest distances
// from a source vertex to all other vertices
vector<int> BellmanFord_Algorithm(const vector<vector<int>> &edges, int V) {
    vector<int> dist(V + 1, INF); // Distance from source to each vertex
    dist[V] = 0; // Distance to the new source vertex (added vertex) is 0

    // Add a new source vertex to the graph and connect it to all original
    // vertices with 0 weight edges
    vector<vector<int>> edges_with_extra(edges);
    for (int i = 0; i < V; ++i) {
        edges_with_extra.push_back({V, i, 0});
    }

    // Relax all edges |V| - 1 times
    for (int i = 0; i < V; ++i) {
        for (const auto &edge : edges_with_extra) {
            if (dist[edge[0]] != INF && dist[edge[0]] + edge[2] < dist[edge[1]]) {
                dist[edge[1]] = dist[edge[0]] + edge[2];
            }
        }
    }
    return vector<int>(dist.begin(),
                      dist.begin() +
                      V); // Return distances excluding the new source vertex
}

// Function to implement Johnson's Algorithm
void JohnsonAlgorithm(const vector<vector<int>> &graph) {
    int V = graph.size(); // Number of vertices
    vector<vector<int>> edges;

    // Collect all edges from the graph
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (graph[i][j] != 0) {
                edges.push_back({i, j, graph[i][j]});
            }
        }
    }
}

```

```

}

// Get the modified weights from Bellman-Ford algorithm
vector<int> altered_weights = BellmanFord_Algorithm(edges, V);
vector<vector<int>> altered_graph(V, vector<int>(V, 0));

// Modify the weights of the edges to remove negative weights
for (int i = 0; i < V; ++i) {
    for (int j = 0; j < V; ++j) {
        if (graph[i][j] != 0) {
            altered_graph[i][j] =
                graph[i][j] + altered_weights[i] - altered_weights[j];
        }
    }
}

// Print the modified graph with re-weighted edges
cout << "Modified Graph:\n";
for (const auto &row : altered_graph) {
    for (int weight : row) {
        cout << weight << ' ';
    }
    cout << endl;
}

// Run Dijkstra's algorithm for every vertex as the source
for (int source = 0; source < V; ++source) {
    cout << "\nShortest Distance with vertex " << source << " as the source:\n";
    Dijkstra_Algorithm(graph, altered_graph, source);
}

// Main function to test the Johnson's Algorithm implementation
int main() {
    // Define a graph with possible negative weights
    vector<vector<int>> graph = {
        {0, -5, 2, 3}, {0, 0, 4, 0}, {0, 0, 0, 1}, {0, 0, 0, 0}};

    // Execute Johnson's Algorithm
    JohnsonAlgorithm(graph);
    return 0;
}

```

Johnsons ALogrithm Scheduling

```
#include <bits/stdc++.h>
```

```

using namespace std;
using ll = long long;
using vi = vector<int>;
using vll = vector<ll>;

#define REP(i, a, b) for (int i = (a); i < (b); ++i)
#define ALL(x) (x).begin(), (x).end()

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    if (!(cin >> N))
        return 0;
    vector<ll> p1(N), p2(N);
    for (int i = 0; i < N; ++i)
        cin >> p1[i] >> p2[i];

    // Johnson's algorithm
    vector<int> A(N);
    iota(A.begin(), A.end(), 0); // remaining job indices
    deque<int> L2;
    vector<int> L1;
    vector<char> used(N, 0);

    int remaining = N;
    while (remaining > 0) {
        // find min among available p1[i], p2[i] where not used
        ll mn = LLONG_MAX;
        int which = -1; // job index
        int side = 0; // 1 => belongs to p1, 2 => belongs to p2
        for (int i = 0; i < N; ++i) {
            if (used[i])
                continue;
            if (p1[i] < mn) {
                mn = p1[i];
                which = i;
                side = 1;
            }
            if (p2[i] < mn) {
                mn = p2[i];
                which = i;
                side = 2;
            }
        }
        // tie-breaking: above loop picks first found minimum (smallest index)
        if (which == -1)
            break;
        used[which] = 1;
        --remaining;
        if (side == 1)
            L1.push_back(which);
        else
    
```

```

    L2.push_front(which);
}

// final sequence: L1 followed by L2
vector<int> seq;
seq.reserve(N);
for (int x : L1)
    seq.push_back(x);
for (int x : L2)
    seq.push_back(x);

// simulate to compute makespan and optional completion times
ll t1 = 0, t2 = 0;
for (int idx : seq) {
    t1 += p1[idx]; // finish time on M1 for this job
    if (t2 < t1)
        t2 = t1; // M2 can only start after M1 finishes for that job and after M2
                  // is free
    t2 += p2[idx]; // finish time on M2 for this job
}
ll makespan = t2;

// Output: sequence (1-based indices) and makespan
for (int i = 0; i < (int)seq.size(); ++i) {
    if (i)
        cout << ' ';
    cout << (seq[i] + 1);
}
cout << '\n' << makespan << '\n';
return 0;
}

```

Maximal Independent Set

```

#include <iostream>
#include <vector>
using namespace std;

const int MAXN = 1005; // Adjust based on constraints
bool adj[MAXN][MAXN];
bool active[MAXN];

vector<int> findMIS(int n) {
    vector<int> mis;

    for (int i = 0; i < n; i++) {
        if (!active[i])

```

```
        continue;

    mis.push_back(i);
    active[i] = false;

    // Remove all neighbors of i
    for (int j = 0; j < n; j++) {
        if (adj[i][j]) {
            active[j] = false;
        }
    }
}

return mis;
}

int main() {
    int n, m;
    cin >> n >> m;

    // Initialize adjacency matrix
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u][v] = adj[v][u] = true;
    }

    // Initialize all nodes as active
    for (int i = 0; i < n; i++) {
        active[i] = true;
    }

    vector<int> mis = findMIS(n);

    // Output the result
    cout << mis.size() << endl;
    for (int node : mis) {
        cout << node << " ";
    }
    cout << endl;

    return 0;
}
```

Minimum Clique Cover

```

#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> g;

vector<int> match, p, base;
vector<char> used, blossom;
vector<int> q;

int lca(int a, int b) {
    vector<char> used_local(n, false);
    while (true) {
        a = base[a];
        used_local[a] = true;
        if (match[a] == -1)
            break;
        a = p[match[a]];
    }
    while (true) {
        b = base[b];
        if (used_local[b])
            return b;
        b = p[match[b]];
    }
}

void mark_path(int v, int b, int children) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] = true;
        p[v] = children;
        children = match[v];
        v = p[match[v]];
    }
}

int find_path(int root) {
    used.assign(n, false);
    p.assign(n, -1);
    for (int i = 0; i < n; ++i)
        base[i] = i;

    q.clear();
    q.push_back(root);
    used[root] = true;
    size_t qh = 0;
    while (qh < q.size()) {
        int v = q[qh++];
        for (int to : g[v]) {
            if (base[v] == base[to] || match[v] == to)
                continue;
            if (to == root || (match[to] != -1 && p[match[to]] != -1)) {

```

```

    int cur = lca(v, to);
    blossom.assign(n, false);
    mark_path(v, cur, to);
    mark_path(to, cur, v);
    for (int i = 0; i < n; ++i) {
        if (blossom[base[i]]) {
            base[i] = cur;
            if (!used[i]) {
                used[i] = true;
                q.push_back(i);
            }
        }
    }
} else if (p[to] == -1) {
    p[to] = v;
    if (match[to] == -1)
        return to;
    int nxt = match[to];
    used[nxt] = true;
    q.push_back(nxt);
}
}
return -1;
}

void augment_path(int finish) {
    int v = finish;
    while (v != -1) {
        int pv = p[v];
        int nv = (pv == -1 ? -1 : match[pv]);
        match[v] = pv;
        if (pv != -1)
            match[pv] = v;
        v = nv;
    }
}

int max_matching() {
    match.assign(n, -1);
    base.resize(n);
    int res = 0;
    for (int i = 0; i < n; ++i) {
        if (match[i] == -1) {
            int v = find_path(i);
            if (v == -1)
                continue;
            ++res;
            augment_path(v);
        }
    }
    return res;
}

```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    if (!(cin >> n >> m))
        return 0;
    g.assign(n, {});
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        // assume input is 0-indexed. If input is 1-indexed, subtract 1 here.
        if (u == v)
            continue;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int matching_size = max_matching();
    int min_clique_cover = n - matching_size;
    cout << min_clique_cover << '\n';

    vector<char> used_out(n, false);
    // print matched pairs (each pair once) and singletons
    for (int v = 0; v < n; ++v) {
        if (match[v] != -1 && v < match[v]) {
            cout << 2 << ' ' << v << ' ' << match[v] << '\n';
            used_out[v] = used_out[match[v]] = true;
        }
    }
    for (int v = 0; v < n; ++v) {
        if (!used_out[v]) {
            cout << 1 << ' ' << v << '\n';
        }
    }

    return 0;
}
```
